

Garner's algorithm

A consequence of the [Chinese Remainder Theorem](#) is, that we can represent big numbers using an array of small integers. For example, let p be the product of the first 1000 primes. p has around 3000 digits.

Any number a less than p can be represented as an array $a_1, \dots, a_k$, where $a_i \equiv a \pmod{p_i}$. But to do this we obviously need to know how to get back the number a from its representation. One way is discussed in the article about the Chinese Remainder Theorem.

In this article we discuss an alternative, Garner's Algorithm, which can also be used for this purpose.

Mixed Radix Representation

We can represent the number a in the **mixed radix** representation:

$$a = x_1 + x_2 p_1 + x_3 p_1 p_2 + \dots + x_k p_1 \dots p_{k-1} \text{ with } x_i \in [0, p_i)$$

A mixed radix representation is a positional numeral system, that's a generalization of the typical number systems, like the binary numeral system or the decimal numeral system. For instance the decimal numeral system is a positional numeral system with the radix (or base) 10. Every a number is represented as a string of digits $d_1 d_2 d_3 \dots d_n$ between 0 and 9, and E.g. the string 415 represents the number $4 \cdot 10^2 + 1 \cdot 10^1 + 5 \cdot 10^0$. In general the string of digits $d_1 d_2 d_3 \dots d_n$ represents the number $d_1 b^{n-1} + d_2 b^{n-2} + \dots + d_n b^0$ in the positional numeral system with radix b .

In a mixed radix system, we don't have one radix any more. The base varies from position to position.

Garner's algorithm

Garner's algorithm computes the digits $x_1, \dots, x_k$. Notice, that the digits are relatively small. The digit x_i is an integer between 0 and $p_i - 1$.

Let r_{ij} denote the inverse of p_i modulo p_j

$$r_{ij} = (p_i)^{-1} \pmod{p_j}$$